JavaScript Asinkron: fetch, Promise, dan JSON

Pengembangan Aplikasi Berbasis Web · SIF302 · Semester Gasal 2026/2027

NAMA: ILHAM AJI KUSUMA	NIM: 25523172	KELAS: D	TANGGAL: 15/10/2026
WORKSHEET	**P10**	**Pertemuan 10 · JavaScript Asinkron: fetch, Promise, dan JSON**	

Dikerjakan sendiri, bukan berpasangan. Setiap mahasiswa mengerjakan satu halaman profil miliknya sendiri. Anda bebas memilih isi halaman, tetapi harus bisa menjelaskan setiap baris kode yang Anda tulis.

Capaian pembelajaran	Yang dinilai hari ini
Mahasiswa mampu mengambil data dari sebuah API memakai fetch, membaca balasan JSON, memisahkan lapisan data dari lapisan tampilan dalam berkas js/api.js, lalu menampilkan hasilnya di halaman beserta keadaan memuat, berhasil, gagal, dan kosong.	1. Pemanggilan fetch benar dan alamatnya ditetapkan di satu tempat. 2. respons.ok diperiksa sebelum badan balasan dibaca. 3. JSON.parse dan JSON.stringify dipakai pada arah yang tepat. 4. Keempat keadaan antarmuka tampil: memuat, berhasil, gagal, kosong. 5. Galat ditangkap dan pesannya ditampilkan di halaman, tidak sekadar di console. 6. Berkas js/api.js memuat lapisan data dan mengekspor fungsinya, dan app.js memanggilnya tanpa memanggil fetch sendiri.

Aturan pengerjaan

Hal	Ketetapan
Bentuk kerja	Individu. Satu mahasiswa, satu halaman, satu topik.
Perkakas	VS Code + peramban Chrome atau Firefox. Panel Console dan Network di DevTools dipakai terus-menerus mulai lembar A.
Berkas awal	Hasil pekerjaan Anda sendiri dari Pertemuan 9, profil.html, seluruh berkas CSS, js/app.js, dan js/dom.js, disalin ke folder worksheet-p10/. Tidak ada berkas contoh, tidak ada kerangka yang disediakan.
Berkas baru	Dua berkas baru: data/proyek.json berisi data proyek Anda, dan js/api.js yang mengambil data itu lalu mengekspor fungsinya. Satu berkas lama disentuh: js/app.js, ditambah impor, pemanggilan, dan try/catch. js/dom.js dari Pertemuan 9 tidak berubah perannya: seluruh sentuhan ke halaman tetap di sana, termasuk render().
API	Data Anda sendiri, dari dua tempat. Mula-mula file data/proyek.json di folder Anda, dikirim oleh Live Server. Di lembar D.5, file yang sama diambil dari repositori GitHub Anda lewat alamat raw.githubusercontent.com. Tidak perlu API key.
Waktu	90 menit di kelas. Bila belum selesai, dilanjutkan di luar kelas paling lambat pukul 23.59 pada hari yang sama.
Penyerahan	Seluruh isi worksheet-p10/ diunggah ke repositori GitHub Anda sendiri, di dalam folder semester ~/pabw/. Periksa halaman repositori Anda, tidak cukup folder di laptop.

Anggaran waktu 90 menit

Menit	Lembar	Yang Anda kerjakan	Kalau waktu habis, berhenti di sini
0-18	A	Salin hasil Pertemuan 9, pindahkan data ke data/proyek.json, buat js/api.js, fetch pertama, periksa respons.ok	Objek Response tampil di Console
18-36	B	Ubah rantai then menjadi async/await, satu try/catch, buktikan galatnya tertangkap	Galat jaringan tertangkap dan tampil di halaman
36-54	C	Baca balasan dengan json(), petakan bentuk datanya, kenali arah stringify	Kunci data teridentifikasi dan terdokumentasi
54-74	D	Telusuri larik, gambar kartu ke halaman, keempat keadaan antarmuka, ambil data dari GitHub	Daftar tampil di halaman dari data
74-96	E	Tiga kasus sulit: 404 dianggap berhasil, await di fungsi yang bukan async, balasan tanpa butir	Tiga kasus ditangani + catatan
96-150	F	Periksa satu per satu, satu baris untuk diingat, tiket keluar, penilaian mandiri, README, catat bagian P13	Lembar F terisi dan tersimpan

Selalu kerjakan berurutan. Setiap lembar mengandaikan lembar sebelumnya sudah selesai, dan setiap keadaan antarmuka dibuktikan dengan tangkapan layar. Kolom paling kanan menunjukkan laju, bukan tenggat.

WORKSHEET **A** **Panggilan pertama dan status balasan**

fetch TIDAK MELEMPAR GALAT PADA STATUS 404, PERIKSA respons.ok

Panggilan pertama sering dianggap berhasil karena tidak ada galat di Console. Padahal balasan bisa berstatus 404 atau 500 dan tetap dianggap berhasil oleh fetch. Satu pemeriksaan singkat menutup seluruh celah itu.

A.1 Berkas yang harus ada sebelum Anda mulai

Salin seluruh hasil Pertemuan 9 ke folder worksheet-p10/. Periksa satu per satu, pekerjaan hari ini bertumpu pada berkas-berkas itu, dan yang hilang akan terasa sebagai galat 404 di Console.

Berkas	Dari	Ada?	Jumlah baris
profil.html	P9	Ada	126
Seluruh berkas CSS (enam berkas)	P4-P6	Ada	Base: 27, komponen: 153, layout: 56, respon: 27, tema: 28, token: 35, total: 326 baris
js/app.js	P8	ada	59
js/dom.js	P9	ada	142

A.2 Siapkan elemen untuk keadaan memuat

Wadah daftar dan pesan daftar kosong sudah Anda buat di Pertemuan 9, jadi cukup diperiksa apakah masih ada. Yang baru hari ini hanya satu: pesan “sedang memuat”. Tambahkan di profil.html sekarang, sebelum menulis JavaScript. Nama id boleh Anda pilih sendiri, asalkan sama huruf per huruf dengan yang Anda tulis di querySelector nanti.

```
<!-- dari Pertemuan 9: cukup dicek masih ada -->
<ul id="daftar"></ul>                <!-- wadah kartu <li> -->
<p id="kosong" hidden>Belum ada data.</p>
<!-- baru hari ini -->
<p id="muat" hidden>Memuat data...</p>
```

Atribut hidden wajib ada pada pesan memuat. Tanpa hidden, pesan itu tampil sejak halaman dibuka, sebelum data diminta.

Nama #daftar, #kosong, dan #muat dipakai di seluruh contoh lembar ini. Bila di Pertemuan 9 Anda memakai nama lain, pakai nama Anda sendiri di setiap tempat: di HTML, di querySelector, dan di tabel D.3.

A.3 Pindahkan data ke data/proyek.json, lalu tetapkan alamatnya

Daftar proyek yang selama ini ditulis sebagai array di app.js dipindahkan ke file data/proyek.json. Inilah "API" pertama Anda: file JSON yang dikirim server lokal (Live Server) setiap kali diminta. Tulis ulang isinya mengikuti aturan JSON: kunci dan teks memakai tanda kutip ganda, tanpa koma di akhir, tanpa komentar. Pastikan setiap proyek punya id yang unik, karena id dipakai lagi di Pertemuan 11. Alamatnya ditulis satu kali saja, di bagian atas js/api.js.

```
// data/proyek.json  (pakai kunci data Anda sendiri)
[
  { "id": 1, "judul": "Halaman Profil", "tahun": 2026, "kategori": "web" },
  { "id": 2, "judul": "Katalog Produk", "tahun": 2026, "kategori": "web" }
]

// js/api.js, bagian atas file
const ALAMAT = 'data/proyek.json';   // relatif terhadap profil.html
```

Hal yang diperiksa	Tanda benar
Alamat	'data/proyek.json', relatif terhadap profil.html
Satu tempat	alamat tidak ditulis ulang di fungsi lain
Tanpa kunci akses	tidak ada apiKey atau token di dalam kode
Nama berkas	alamat berada di js/api.js, bukan di js/app.js

A.4 Panggil, periksa, lalu buktikan di panel Network

Panggil fetch, lalu periksa respons.ok sebelum membaca badannya. Isi tabel berikut dari panggilan Anda sendiri.

```
async function muat() {
  const respons = await fetch(ALAMAT);
  console.log('status', respons.status, 'ok', respons.ok);
  if (!respons.ok) {
    throw new Error('status ' + respons.status);
  }
  return respons.json();
}
```

Yang dicatat	Nilai yang saya dapat
Alamat API yang saya pakai	./data/proyek.json
Status balasan (200, 404, dan seterusnya)	200
Nilai respons.ok	True
Jenis isi (Content-Type)	Lihat di Network

Buka satu panggilan di panel Network. Periksa tab Headers untuk status dan jenis isi, tab Response untuk badan mentahnya. Badan mentah inilah yang nanti dibaca json(), jadi kenali bentuknya sebelum menulis kode penggambar.

SIMPAN KE GIT - SETELAH LEMBAR A

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: panggilan pertama dan status balasan
```

Lembar A sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

A - simpan perubahan

Cara memastikan: buka Console, panggil fungsi muat() Anda. Status dan nilai respons.ok tampil, dan tidak ada tulisan merah.

WORKSHEET **B** **async, await, dan penanganan galat**

SATU try/catch MENUTUP SELURUH RANTAI

Rantai then yang panjang sulit dibaca dan galatnya mudah terlewat. async dan await menuliskan alur yang sama secara mendatar, dan satu blok try/catch menangkap galat dari seluruh langkah di dalamnya.

B.1 Ubah rantai menjadi async/await

Tulis fungsi async, lalu bungkus seluruh langkahnya dengan try/catch. Bandingkan bentuknya dengan rantai then yang Anda tulis di Lembar A.

```
export async function ambilProyek() {
  try {
    const respons = await fetch(ALAMAT);
    if (!respons.ok) throw new Error('status ' + respons.status);
    const data = await respons.json();
    return data;
  } catch (galat) {
    console.error('gagal memuat', galat.message);
    throw galat;
  }
}
```

await hanya boleh dipakai di dalam fungsi async. Pemanggil di luar fungsi itu masih perlu menangani penolakannya, di pekerjaan Anda, pemanggilnya adalah app.js, dan di sanalah blok try/catch kedua dipasang.

B.2 Kenali tiga kegagalan

Jenis kegagalan	Yang terjadi pada Promise	Cara menanganinya
Jaringan mati atau alamat salah	Promise ditolak	catch menangkap galat, pesannya ditampilkan
Status 404 atau 500	Promise tetap berhasil	periksa respons.ok, lalu lempar galat sendiri
Badan bukan JSON	json() menolak	catch menangkap galat, pesannya ditampilkan
Balasan sah tetapi tanpa butir	Promise berhasil, larik kosong	tampilkan keadaan kosong, jangan layar putih

B.3 Buktikan galatnya tertangkap

Yang saya uji	Hasil yang benar	Hasil yang saya dapat
Alamat diubah menjadi alamat yang tidak ada	Pesan galat tampil di halaman, Console bersih dari galat tak tertangani	Pesan “Data gagal dimuat…” tampil di halaman, proyk.json 404 di Console, tanpa Uncaught
Jaringan dimatikan di panel Network	Pesan galat tampil, bukan layar kosong	Halaman “No Internet” milik browser, profil.html sendiri berstatus failed
Alamat dikembalikan seperti semula	Daftar tampil kembali tanpa menyunting kode lain	proyek.json 200 (type fetch, initiator api.js:5), pesan galat hilang

Sengaja rusakkan, buktikan, lalu kembalikan. Uji yang tidak dikembalikan akan tampak sebagai kerusakan pada pertemuan berikutnya, dan Anda yang akan menelusurinya kembali.

SIMPAN KE GIT - SETELAH LEMBAR B

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: async, await, dan satu try/catch
```

Lembar B sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

B - simpan perubahan

Cara memastikan: ubah alamat menjadi alamat yang salah, muat ulang halaman. Pesan galat tampil di halaman, dan Console tidak menampilkan galat tak tertangani.

WORKSHEET **C** **Membaca JSON**

TEKS MASUK, NILAI KELUAR, DAN SEBALIKNYA

Balasan jaringan datang sebagai teks. json() mengubahnya menjadi nilai JavaScript sebelum dapat dipakai. Arah sebaliknya dipakai saat mengirim data ke peladen, dan di situlah JSON.stringify bekerja.

C.1 Dua arah

Baca badan balasan dengan json(), dan kirim data dengan stringify. Perhatikan jenis nilai yang keluar pada tiap arah, salah arah adalah penyebab undefined yang paling sering.

```
// teks menjadi objek
const data = await respons.json();
console.log(data[0].judul);   // data berupa larik: lihat butir pertama

// objek menjadi teks
const badan = JSON.stringify({ judul: 'Judul contoh', kategori: 'web' });
console.log(typeof badan);   // string
```

Yang dipakai	Arah	Hasilnya	Kapan dipakai
respons.json()	teks → nilai	bentuk data yang bisa diakses dengan kunci	membaca balasan peladen
JSON.parse(teks)	teks → nilai	sama, tetapi untuk teks yang Anda pegang sendiri	membaca localStorage atau berkas JSON lokal
JSON.stringify(nilai)	nilai → teks	teks yang siap dikirim atau disimpan	mengirim data, menyimpan ke localStorage

C.2 Peta data saya

Tulis bentuk balasan API Anda sendiri. Kolom kiri adalah nama kunci, kolom tengah jenis nilainya dan contoh isinya, kolom kanan cara mengaksesnya di dalam kode.

Nama kunci	Jenis nilai dan contoh	Cara mengaksesnya
akar	larik berisi objek - 3 anime	data.length, data.forEach(...)
id	Angka - 1	data[0].id
judul	Teks – “wistoria wand and sword”	Data[0].judul
Season	Angka - 2	Data[0].season
episode	Angka – 24	Data[0].episode
Kategori	Larik teks - [“action”, “fantasy”]	data[0].kategori[0]
gambar	Teks - “thumb-1920-772376.jpg”	data[1].gambar
alt	Teks - “Logo anime One Piece...”	data[2].alt

C.3 Periksa bentuknya sebelum menggambar

Panggil console.log(data) sekali sebelum menulis kode penggambar. Membaca bentuk data lebih dulu menghemat waktu, karena kunci yang salah hanya menghasilkan undefined tanpa galat yang jelas.

SIMPAN KE GIT - SETELAH LEMBAR C

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: membaca JSON dua arah
```

Lembar C sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

C - simpan perubahan

Cara memastikan: tulis nama kunci Anda di Console satu per satu. Tidak ada satu pun yang bernilai undefined.

WORKSHEET **D** **Menampilkan data ke halaman**

PISAHKAN PENGAMBILAN DATA DARI PENGGAMBARAN

Fungsi pengambil data hanya bertugas mengambil dan memeriksa. Fungsi penggambar hanya bertugas menyusun elemen. Dipisah begitu, halaman mudah diuji ketika alamat peladennya berubah, dan itulah bentuk yang dipakai lagi di Pertemuan 11.

D.1 Perluas render() dari Pertemuan 9

Fungsi render() sudah Anda tulis di Pertemuan 9. Hari ini ia menerima sumber data yang berbeda, bukan tugas yang berbeda: larik hasil parse dari jaringan, bukan larik dari app.js. Jangan membuat fungsi penggambar kedua. Gunakan textContent, bukan innerHTML, untuk isi yang datang dari jaringan. Kartunya juga tetap dibuat oleh buatKartu() dari Pertemuan 9. Bila kunci data dari API berbeda dengan data Anda (misalnya title, bukan judul), sesuaikan di buatKartu(), bukan di render().

```
// js/dom.js — render() dari Pertemuan 9, diperluas hari ini
export function render(daftar) {
  wadah.textContent = '';           // 1. kosongkan lebih dulu
  if (daftar.length === 0) {        // 2. periksa keadaan kosong
    kosong.textContent = 'Belum ada data.';
    kosong.hidden = false;
    return;
  }
  kosong.hidden = true;
  daftar.forEach((proyek) => wadah.append(buatKartu(proyek)));   // 3. isi ulang
}
```

D.2 Taruh di berkas yang benar

Pekerjaan hari ini menyentuh tiga berkas, dan pembagiannya tidak boleh tertukar. js/api.js mengambil data dan memeriksa statusnya. js/dom.js menyentuh halaman: memilih elemen, menggambar kartu, menampilkan pesan. js/app.js merakit keduanya. Aturannya satu: kata fetch hanya boleh muncul di js/api.js.

```
// js/app.js — perakit (tiga baris baru, sisanya dari Pertemuan 8)
import { ambilProyek } from './api.js';
import { render, tampilkanGalat, aturMemuat } from './dom.js';

async function muat() {
  aturMemuat(true);
  try {
    const data = await ambilProyek();     // respons.ok sudah diperiksa di api.js
    render(data);                         // larik mentah, bukan HTML
  } catch (galat) {
    tampilkanGalat('Data gagal dimuat. Periksa koneksi, lalu muat ulang halaman.');
    console.error(galat.message);         // untuk Anda, bukan untuk pengguna
  } finally {
    aturMemuat(false);
  }
}

muat();
```

Penyaring kategori dari Pertemuan 9 tetap berlaku. render() menerima larik yang sudah disaring; jangan menggambar seluruh data mentah langsung dari jaringan. Di Pertemuan 11 penyaring itu dipindah ke data.js sebagai fungsi saring().

Tiga nama yang diekspor dari js/dom.js — render, tampilkanGalat, dan aturMemuat — adalah kontrak berkas itu. Pertemuan 11 akan mengimpor render dari sana, jadi jangan mengganti namanya tanpa mengganti seluruh pemanggilnya.

D.3 Empat keadaan antarmuka

Isi tabel berikut dengan cara Anda menampilkan tiap keadaan. Keadaan kosong sering terlupa, padahal balasan yang sah bisa berisi larik kosong.

Keadaan	Kapan terjadi	Apa yang tampil di halaman
Memuat	sebelum balasan datang	“memuat data...”
Berhasil	balasan berisi data	3 kartu
Gagal	jaringan gagal atau status bukan 2xx	“Data gagal dimuat. Periksa koneksi, lalu muat ulang halaman.”
Kosong	balasan sah tetapi tanpa butir	“Belum ada data.” (atau “Belum ada anime di kategori ini.” saat filter, mis. Romance)

D.4 Periksa batas lapisan

Yang diperiksa	Hasil yang benar	Hasil yang saya dapat
js/api.js memanggil fetch dan mengekspor fungsinya	satu export terlihat di bagian atas berkas	ambilProyek() di js/api.js
js/app.js tidak memanggil fetch sendiri	kata fetch hanya ada di js/api.js	Pencarian “fetch” hanya cocok di js/api.js
app.js menangani penolakan dari api.js	blok try/catch kedua ada di pemanggil	Di muat() pada app.js
Wadah dikosongkan sebelum digambar ulang	jumlah kartu tidak berlipat saat dimuat ulang	wadah.textContent = "" di baris pertama render(). Kartu tidak menumpuk saat filter berpindah
Nama fungsi yang diekspor tercatat	nama itu yang akan ditulis di berkas pengimpor Pertemuan 11	ambilProyek (api.js); render, tampilkanGalat, aturMemuat, tampilkanData (dom.js)
Pemisahan tiga berkas	api.js mengambil, dom.js menggambar, app.js merakit	api.js mengambil data, dom.js menggambar, app.js merakit
Data diambil dari GitHub (D.5)	panel Network: raw.githubusercontent.com, status 200	**[ISI]** setelah push

D.5 Ambil data yang sama dari GitHub

Setelah semuanya bekerja dengan file lokal, ambil data yang sama dari internet. File data/proyek.json sudah ikut ter-push ke GitHub sejak lembar A. GitHub menyediakan alamat "raw" untuk setiap file: isi file apa adanya, tanpa tampilan halaman GitHub, dan alamat itu boleh diambil dengan fetch dari browser. Cukup ganti satu baris ALAMAT di js/api.js. Tidak ada kode lain yang berubah, dan itulah gunanya menulis alamat di satu tempat.

```
// buka proyek.json di github.com, klik tombol Raw,
// lalu salin alamat dari address bar browser
const ALAMAT =
  'https://raw.githubusercontent.com/' +
  'username/nama-repo/main/worksheet-p10/data/proyek.json';
```

Perhatikan tiga hal. Pertama, repositori harus publik; repositori privat menjawab 404. Kedua, sesuaikan nama cabang (main atau master) dan jalur foldernya huruf per huruf. Ketiga, setelah push, isi baru di alamat raw bisa terlambat sampai sekitar lima menit karena disimpan sementara (cache). Buka panel Network dan pastikan permintaan sekarang menuju raw.githubusercontent.com dengan status 200. Biarkan alamat GitHub ini sebagai hasil akhir; bila jaringan kelas bermasalah, kembalikan ke data/proyek.json dan catat alasannya di README.

SIMPAN KE GIT - SETELAH LEMBAR D

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: data dari jaringan tampil di halaman
```

Lembar D sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

D - simpan perubahan

Cara memastikan: muat ulang halaman tanpa menyentuh Console. Kartu tampil dari data, dan tidak ada kartu yang berlipat.

WORKSHEET **E** **Tiga kasus sulit**

TIDAK ADA GALAT DI CONSOLE, TETAPI HALAMAN DIAM

Ketiga kasus berikut tidak selalu menimbulkan galat. Yang terlihat hanya halaman yang diam atau kosong, sehingga gejalanya perlu dibaca lebih dulu sebelum kodenya disentuh. Kasusnya bisa Anda buat sendiri di file lokal: salah ketik nama file (data/proyk.json) untuk 404, isi file diganti [] untuk keadaan kosong, dan satu tanda kutip dihapus dari proyek.json untuk SyntaxError. Kembalikan seperti semula setelah diuji.

E.1 Status 404 tidak melempar galat

fetch hanya menolak Promise ketika permintaannya gagal, jaringan mati, alamat tidak dapat dijangkau, atau permintaan diblokir. Status 404 dan 500 tetap dianggap berhasil. Satu pemeriksaan respons.ok menutup celah itu sebelum badannya dibaca.

Gejala	Sebab yang paling sering	Yang dilakukan
Halaman menampilkan kartu kosong	badan 404 dibaca sebagai data	periksa respons.ok sebelum memanggil json()
Daftar berisi undefined	kunci data salah, tetapi statusnya 200	cetak console.log(data) sekali untuk melihat bentuknya
json() melempar SyntaxError	badan bukan JSON, sering halaman galat HTML	periksa Content-Type di panel Network

E.2 await di dalam fungsi yang bukan async

await hanya sah di dalam fungsi yang diberi kata async. Di file type="module", await di baris paling luar memang diperbolehkan, jadi kesalahan yang sering terjadi ada di tempat lain: await dipakai di dalam fungsi biasa, misalnya callback addEventListener. Akibatnya file gagal dibaca browser dan seluruh skrip berhenti, bukan hanya satu baris. Console menulis: await is only valid in async functions and the top level bodies of modules.

```
// salah: callback biasa, bukan async
tombol.addEventListener('click', function () {
  const data = await ambilProyek();
});

// benar: callback diberi async
tombol.addEventListener('click', async function () {
  const data = await ambilProyek();
});
```

E.3 Balasan sah tetapi tanpa butir

Balasan dengan larik kosong tidak menimbulkan galat apa pun. Halaman hanya tampak kosong, dan pengguna tidak tahu apakah datanya belum ada atau kodenya salah. Keadaan kosong perlu ditulis sebagai keadaan tersendiri.

Tulis pesan kosong yang menjelaskan keadaan yang sedang terjadi. "Belum ada proyek" lebih berguna daripada halaman putih, dan itu yang membedakan halaman yang selesai dari halaman yang setengah jadi.

E.4 Catat kasus yang saya temui

Gejala yang saya lihat	Sebabnya	Cara saya memperbaiki
Nama file salah (proyk.json): tidak ada kartu, 404 di Console, tapi fetch tidak error	fetch hanya menolak kalau permintaan gagal terkirim. 404 tetap balasan HTTP yang sah	if (!respons.ok) throw new Error("status " + respons.status) sebelum json()
Isi file []: halaman tanpa kartu, Console bersih	Balasan sah tapi tanpa butir	Cabang daftar.length === 0 di render() menampilkan “Belum ada data.”
Satu tanda kutip dihapus dari proyek.json: SyntaxError	Badan balasan bukan JSON valid, json() menolak	catch di api.js mencatat lalu melempar, catch di app.js menampilkan pesan di halaman

SIMPAN KE GIT - SETELAH LEMBAR E

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: tiga kasus sulit ditangani
```

Lembar E sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

E - simpan perubahan

Cara memastikan: rusakkan alamat, matikan jaringan, dan kosongkan data. Ketiga keadaan tampil sebagai pesan di halaman, bukan layar putih.

WORKSHEET **F** **Periksa, tiket keluar, penilaian mandiri**

SETIAP KEADAAN DIBUKTIKAN, CARA PEMBUKTIAN BEBAS

Pemeriksaan dilakukan pada keempat keadaan, dan setiap keadaan dibuktikan dengan cara yang bisa Anda ulangi di depan pengampu: tangkapan layar, rekaman layar, atau berkas hasil render. Untuk keadaan berhasil, sertakan juga berkas HTML hasil render supaya terlihat datanya benar-benar menjadi elemen halaman. Tanpa bukti itu, halaman yang hanya bekerja sekali sulit ditelusuri kembali.

F.1 Periksa satu per satu

Periksa	Cara memeriksa	Lolos
Alamat di satu tempat	lihat bagian atas js/api.js	✓
respons.ok diperiksa	lihat fungsi pengambil di js/api.js	✓
Galat ditangkap	matikan jaringan, muat ulang halaman	✓
Keadaan memuat tampil	perlambat jaringan di panel Network	✓
Keadaan kosong tampil	panggil render([]) dari Console	✓
Data tampil sebagai elemen	periksa wadah di panel Elements	✓
Batas lapisan terjaga	kata fetch hanya ada di js/api.js	✓
Tidak ada galat di Console	baca panel Console	✓
Pengungkapan AI	README menyebutkan bagian mana yang dibantu AI dan bagian mana yang saya kerjakan sendiri	✓
Keaslian	topik dan isi halaman ini milik saya sendiri, tidak menyalin pekerjaan mahasiswa lain	✓

F.2 Bukti empat keadaan

Keadaan	Cara saya membuktikan	Catatan
Berhasil	Tangkapan layar halaman dengan tiga kartu	Kartu dibuat dari proyek.json, bukan HTML manual
Gagal	Block request URL pada proyek.json	pesan galat tampil di halaman, tanpa Uncaught di Console. Uji Offline membuat profil.html sendiri gagal dimuat, jadi saya pakai Block request URL
Kosong atau memuat	render([]) di Console, Slow 3G lalu muat ulang	Balasan [] itu sah, jadi render() menampilkan “Belum ada data.”

F.3 Satu baris untuk diingat

Tuliskan satu potongan kode yang paling sering Anda pakai hari ini, dan tuliskan pada bagian mana potongan itu dipakai. Satu baris sudah cukup.

Potongan kode: if (!respons.ok) throw new Error("status " + respons.status);
Dipakai pada: ambilProyek()

F.4 Penilaian mandiri

Bagian	Bobot	Nilai saya	Bukti
Alur data dan pemeriksaan status	25	25	ALAMAT di baris 1 js/api.js, respons.ok diperiksa sebelum json(), ambilProyek diekspor
Pembacaan JSON dan pemisahan lapisan	25	25	respons.json() dan JSON.stringify dipakai (app.js), kata fetch hanya ada di api.js, peta data di C.2

Bagian	Bobot	Nilai saya	Bukti
Keadaan antarmuka	20	20	Memuat, berhasil, gagal, kosong terbukti di folder bukti
Penanganan galat	20	20	Satu try/catch di api.js dan satu di app.js, pesan galat tampil di halaman
Kebersihan kode, deklarasi AI, dan bukti	10	10	Deklarasi AI di README, bukti dicatat di F.2
TOTAL	100	100	

F.5 Tiket keluar

Jawab tiga pertanyaan berikut, satu baris masing-masing. Tiket ini dibaca pengampu sebagai catatan pemahaman Anda.

Pertanyaan	Jawaban
Mengapa fetch tidak melempar galat pada status 404	fetch hanya menolak Promise kalau permintaan gagal terkirim (jaringan putus, alamat tak terjangkau). 404 dan 500 tetap balasan HTTP yang sah, jadi harus diperiksa lewat respons.ok.
Di mana await boleh dipakai, dan di mana tidak	await boleh di dalam fungsi async dan di level paling atas file type="module". Tidak boleh di fungsi biasa, termasuk callback addEventListener yang tidak diberi async.
Satu keadaan antarmuka yang paling mudah terlupa dan cara Anda menutupnya	Keadaan kosong: balasan [] itu sah dan tidak menimbulkan galat, jadi render() punya cabang daftar.length === 0 yang menampilkan “Belum ada data.”

F.6 Catatan untuk pengampu

Bagian yang paling sulit:

Galat “Cannot access ‘daftarAnime’ before initialization”.

Yang ingin saya bahas di kelas berikutnya:

Cara menguji keadaan gagal jaringan dengan benar

Tanda tangan: _____ *Tanggal:* _____ *Jam selesai:* _____

SIMPAN KE GIT - SETELAH LEMBAR F

```
git add .
git commit -m "pesan Anda"
git push
# pesan commit yang disarankan:
# P10: periksa, lalu simpan
```

Lembar F sudah selesai? Simpan hasilnya sekarang, jangan menunggu kelas berakhir. Simpanan ini tidak memerlukan langkah tambahan apa pun, karena fungsinya menjaga agar pekerjaan Anda tidak bergantung pada satu berkas di satu komputer.

F - simpan perubahan

Cara memastikan: buka README di repositori Anda. Bagian Pertemuan 10 sudah memuat deklarasi AI, dan commit terakhir adalah commit terbaru.

WORKSHEET **H** **Lampiran A, rubrik penilaian**

RUBRIK PENILAIAN total 100

Aspek	Bobot	Nilai 100, sangat baik	Nilai 65, memadai	Nilai 35, kurang
Alur data dan pemeriksaan status	25%	Alamat di satu tempat; respons.ok diperiksa sebelum badan dibaca; fungsi pengambil berada di js/api.js dan diekspor	Panggilan berjalan, tetapi respons.ok atau pemisahan ketiga berkas belum lengkap	fetch belum benar, atau status balasan tidak pernah diperiksa
Pembacaan JSON dan pemisahan lapisan	25%	json() dan stringify dipakai pada arah yang tepat; app.js memanggil api.js tanpa memanggil fetch sendiri	Pembacaan benar, tetapi fetch masih ada di app.js atau stringify belum dipakai	Bentuk data ditebak tanpa diperiksa, sehingga banyak nilai undefined
Keadaan antarmuka	20%	Keempat keadaan tampil: memuat, berhasil, gagal, kosong	Tiga keadaan tampil, keadaan kosong belum ada	Hanya keadaan berhasil yang ditangani
Penanganan galat	20%	Satu try/catch menutup seluruh rantai; pesan galat tampil di halaman, tidak sekadar di console	Galat tertangkap, tetapi pesannya hanya di console	Galat tak tertangani, halaman diam tanpa penjelasan
Kebersihan kode, deklarasi AI, dan bukti	10%	Penamaan konsisten, batas api.js dan app.js jelas; deklarasi AI terisi di README; bukti keempat keadaan dicatat di lembar F	Kode rapi tetapi deklarasi AI atau bukti belum lengkap	Deklarasi AI kosong, atau pekerjaan tidak tersimpan di repositori

Yang dinilai adalah pekerjaan Anda sendiri. Topik yang berbeda antar mahasiswa membuat penilaian tidak bisa dibandingkan antar pekerjaan, karena itu yang diperiksa adalah apakah setiap bagian bekerja sebagaimana dimaksud, bukan apakah ia sama dengan contoh.

WORKSHEET H — Lampiran B, rujukan cepat dan kalau macet

REFERENSI CEPAT ASINKRON — hafalkan yang ini

Yang dipakai	Perannya	Kapan dipakai
fetch(url)	Mengirim permintaan HTTP; hasilnya Promise berisi Response	Mengambil proyek.json di ambilProyek() (api.js)
respons.ok	benar bila status 2xx, tidak ada galat otomatis pada 404	Sebelum json(), supaya 404 tidak dibaca sebagai data
respons.status	Kode status balasan: 200, 404, 500, dan seterusnya	Untuk pesan galat "status " + respons.status dan dicek di Network
respons.json()	Membaca badan balasan sebagai JSON; hasilnya Promise	Setelah respons.ok lolos, untuk mendapatkan larik anime
await	Menunggu Promise selesai; hanya di dalam fungsi async	Di depan fetch(...), respons.json(), dan ambilProyek()
async function	Menandai fungsi yang boleh memuat await	ambilProyek() di api.js dan muat() di app.js
try / catch	Menangkap penolakan Promise di sekitarnya	Di api.js (catat lalu lempar) dan di app.js (tampilkan pesan di halaman)
throw new Error()	Membuat galat sendiri, misalnya untuk status bukan 2xx	Saat respons.ok bernilai false, supaya catch menangkapnya
Promise.all([...])	Menunggu beberapa Promise sekaligus	Tidak saya pakai hari ini. Dipakai bila mengambil beberapa file data sekaligus
JSON.parse(teks)	Teks menjadi nilai JavaScript	Membaca teks JSON yang saya pegang sendiri, mis. dari localStorage. respons.json() melakukan hal serupa untuk balasan
JSON.stringify(nilai)	Nilai menjadi teks JSON	Mengubah objek menjadi teks sebelum dikirim, seperti badan di app.js
export / import	Menyerahkan fungsi ke berkas lain, dipakai js/api.js	export di api.js dan dom.js, import di app.js

KALAU MACET — cari gejalanya di sini

Gejala	Penyebab yang paling sering	Yang dilakukan
Console: 404 untuk api.js	Jalur di atribut src tidak cocok dengan letak berkasnya	Pastikan api.js berada di dalam folder js/, huruf besar-kecilnya sama
Console: Cannot read properties of undefined	Kunci data salah, tetapi status balasan 200	Cetak console.log(data) sekali, lalu tulis kunci yang benar-benar ada
Kartu kosong padahal Console bersih	Badan 404 dibaca sebagai data karena respons.ok tidak diperiksa	Periksa respons.ok sebelum memanggil json()
json() melempar SyntaxError	Badan bukan JSON, sering halaman galat HTML	Periksa Content-Type di panel Network
Seluruh berkas berhenti, Console menulis SyntaxError tentang await	await dipakai di fungsi yang bukan async, misalnya callback	Tambahkan kata async pada fungsi itu
Halaman diam tanpa penjelasan	Penolakan Promise tidak ditangkap pemanggil	Tambahkan try/catch di app.js dan tampilkan pesannya di halaman
Daftar kosong tanpa keterangan	Balasan sah berisi larik kosong	Tampilkan keadaan kosong ketika panjang lariknya nol
Kartu berlipat tiap kali dimuat ulang	Wadah tidak dikosongkan sebelum digambar	Tambahkan wadah.textContent = '' di baris pertama penggambar
Isi kartu tampil sebagai tag, bukan teks	Isi dimasukkan dengan innerHTML	Ganti dengan textContent
CORS: permintaan diblokir	Alamat yang dipakai halaman github.com biasa, bukan alamat raw	Pakai alamat dari tombol Raw, atau kembali ke data/proyek.json
404 dari raw.githubusercontent.com	Repositori privat, atau nama cabang dan folder tidak cocok	Jadikan repositori publik, cocokkan jalurnya huruf per huruf

Gejala	Penyebab yang paling sering	Yang dilakukan
Isi lama masih tampil padahal sudah push	Alamat raw disimpan sementara (cache) sekitar lima menit	Tunggu beberapa menit, lalu muat ulang
Fetch gagal hanya di berkas file://	Halaman dibuka dengan klik dua kali	Jalankan lewat server lokal: python3 -m http.server 8000

Halaman yang Anda bangun hari ini adalah bahan ujian CPMK Blok 2 pada Pertemuan 13 nanti: pemilihan elemen, render dari data, event delegation, dan pengambilan data dari jaringan semuanya diuji di sana. Di Pertemuan 11, js/api.js yang Anda buat hari ini diimpor apa adanya, dengan nama fungsi ambilProyek. Pakai nama itu persis, karena nama itulah yang ditulis di berkas pengimpor. Simpan pekerjaan ini rapi di repositori Anda.

LAMPIRAN: MENYIMPAN PEKERJAAN ANDA KE GIT sekali saja, bukan tiap pertemuan

Lampiran ini bukan pengantar Git. Panduan Git dan GitHub yang sudah Anda pakai sejak Pertemuan 3 masih berlaku seluruhnya; bagian ini hanya mengulang tiga perintah yang paling sering dipakai, karena mulai hari ini ada berkas baru yang harus ikut disimpan.

Tiga perintah yang akan Anda pakai terus

Perintah	Artinya	Kapan dijalankan
git add .	Menandai seluruh perubahan di folder ini untuk disimpan	Setiap kali selesai satu lembar
git commit -m "pesan"	Menyimpan perubahan itu sebagai satu riwayat	Langsung setelah git add
git push	Mengirim riwayat itu ke GitHub	Setelah commit, supaya dosen bisa melihatnya

Titik pada git add . Titik itu berarti "seluruh berkas di folder ini". Tanpa titik, Git tidak tahu apa yang harus disimpan. Hal yang sama berlaku untuk berkas api.js yang baru Anda buat, ia ikut tersimpan karena berada di dalam folder semester.

Berapa kali harus menyimpan?

Tidak ada angka yang benar, tetapi patokannya begini: bila Anda sudah menyelesaikan satu hal yang berarti dan bisa Anda jelaskan, simpanlah. Pada pertemuan ini minimal dua kali: setelah data tampil di halaman, dan setelah keempat keadaan antarmuka selesai.

Kalau ini terjadi	Yang Anda lakukan
Sudah menulis banyak, belum pernah commit	Simpan sekarang juga
Salah commit, belum push	Perbaiki lalu commit lagi, jangan hapus repositori
Sudah push, lalu menemukan kekeliruan	Perbaiki berkasnya dan commit lagi; riwayat lama tidak hilang dan itu wajar

Yang dinilai bukan jumlah commit. Menyimpan sering membuat pekerjaan Anda tidak bergantung pada satu berkas di satu komputer, dan pada pertemuan ujian, repositori itulah yang menentukan apakah pekerjaan Anda bisa dinilai.